$$
\begin{array}{rl}
e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid () & \text{constants} \\
\mid \textbf{let}\ x = e_1\ \textbf{in}\ e_2 & \text{let binding} \\
\mid \textbf{uniform}(e_1, e_2) \mid \textbf{gaussian}(e_1, e_2) \mid \textbf{exponential}(e) \mid \ldots & \text{continuous distributions} \\
\mid \textbf{discrete}(p_0, \ldots, p_{n-1}) & \text{discrete distribution} \\
\mid e_1 < e_2 \mid e_1 \le e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \le_{\#n} e_2 & \text{comparisons} \\
\mid e_1 \mathbin{\&\&} e_2 \mid e_1 \mid\mid e_2 \mid \text{not } e & \text{Boolean operations} \\
\mid \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 & \text{conditional} \\
\mid (e_1, e_2) \mid \textbf{fst}\ e \mid \textbf{snd}\ e & \text{pairs} \\
\mid \textbf{fun}\ x \ \to\ e \mid e_1\ e_2 \mid \textbf{fix}\ f\ x := e & \text{functions} \\
\mid \textbf{observe}\ e \mid e_1; e_2 & \text{effects} \\
\mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match}\ e\ \textbf{with} \mid \text{nil} \to e_{\text{nil}} \mid e_1 :: e_2 \to e_{\text{cons}}\ \textbf{end}) & \text{lists} \\
\mid \textbf{diverge} & \text{diverge}
\end{array}
$$

Fig. 5. Syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct $\textbf{observe}(e)$ conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include scalar ordered discrete distributions and continuous distributions with tractable cumulative distribution functions (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). An atomic distribution additionally requires access to CDF left limits. §C lists the distributions implemented by Slice.

The expression $\textbf{discrete}(p_0, \ldots, p_{n-1})$ requires a nonempty probability vector: $n \ge 1$, every $p_i \ge 0$, and $\sum_i p_i = 1$. It returns $k_{\#n}$ with probability p_k.

These are ideal real-valued premises. The implementation separately validates finite binary64 weights. It normalizes compiler-generated weights only within a tight allowance for accumulated CDF arithmetic, and parses source vectors within a documented 10^{-4} rounded-input allowance. These engineering policies do not replace the exact premise above: binary64 evaluation and normalization do not by themselves establish exact preservation of ideal real-valued probabilities, which requires a separate numerical error analysis.

3.2 Types

We formalize the types that Slice supports. A *value* set V is a finite set of reals or $\top$. Elements of $\{<x \mid x \in \mathbb{R}\} \cup \{\le x \mid x \in \mathbb{R}\}$ are called *cuts*. We order cuts first by their real boundary and then by their tag, with $<c \prec \le c$ at a shared boundary c. Thus every finite non-empty cut set has a unique enumeration $B = \{\sim_1 x_1, \ldots, \sim_n x_n\}$ in increasing tagged-cut order. For a cut $b = \sim x$, write $S_b = \{r \in \mathbb{R} \mid r \sim x\}$. The canonical order makes $S_{b_1} \subsetneq \cdots \subsetneq S_{b_n}$. We define the indexed partition induced by B by

$$\text{Intervals}(B)_0 = S_{b_1}, \qquad \text{Intervals}(B)_i = S_{b_{i+1}} \setminus S_{b_i}\ (0 < i < n), \qquad \text{Intervals}(B)_n = \mathbb{R} \setminus S_{b_n}.$$

In particular, $\text{Intervals}(\{<0.1, \le 0.5\}) = \{(-\infty, 0.1), [0.1, 0.5], (0.5, \infty)\}$. The strict and non-strict cuts differ only in which side contains the boundary point: $\text{Intervals}(\{<c\}) = \{(-\infty, c), [c, \infty)\}$, whereas $\text{Intervals}(\{\le c\}) = \{(-\infty, c], (c, \infty)\}$. Thus a boundary generated by $e < c$ places c on the right, while a boundary generated by $e \le c$ places c on the left. Moreover, we let $\text{Intervals}(\emptyset)_0 = \mathbb{R}$;

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
   respective (possibly overlapping) bounds *)
let init =
  (false, within 0.00 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 1.00) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Ensure sampled y hits at most one
   interval *)
let non_overlap =
  fix non_overlap y := fun seen -> fun ys ->
    match ys with
      | nil -> true
      | h :: t ->
          let hit = (snd h) y in
          if hit && seen then false
          else non_overlap y (seen || hit) t
    end in

(* Use the sampled value to update flags *)
let step = fun y -> fun ys ->
  update y ys in

(* Number of samples determined by
   unbounded recursion *)
let cover = fix cover st :=
  if uniform(0,1) < 0.1 then st
  else
    let y = gaussian(0,1) in
    let _ = observe(non_overlap y false st) in
    cover (step y st) in

(* Have all intervals been covered
   at least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run the process and query whether
   the whole interval was covered *)
let final = cover init in
  is_covered final
```

Fig. 21. An infinite stochastic process for random interval covering over $[0, 1]$, involving conditioning, continuous sampling, higher-order recursion, and lists.

B.2 Bloom Filter

The full program for the case study from §7.1.3 is shown in Figure 23. The discretized program is shown in Figure 24.

C List of Supported Slice Distributions

The scalar ordered distributions implemented by Slice are specified below. Continuous rows give densities with respect to Lebesgue measure; discrete rows give probability mass functions with respect to counting measure. A displayed density is zero outside the stated region. The support column gives the topological support, including boundary points that may themselves have zero mass. Parameters are required to be finite in addition to satisfying the displayed domain.

```
(* Set bf[idx] := true, where idx is 0..3 and
   bf = (b0, (b1, (b2, b3))) *)
let set_true = fun bf -> fun idx ->
  let b0 = fst bf in
  let tail1 = snd bf in
  let b1 = fst tail1 in
  let tail2 = snd tail1 in
  let b2 = fst tail2 in
  let b3 = snd tail2 in
  if idx <= 0 then
    (true, (b1, (b2, b3)))
  else if idx <= 1 then
    (b0, (true, (b2, b3)))
  else if idx <= 2 then
    (b0, (b1, (true, b3)))
  else
    (b0, (b1, (b2, true))) in

(* Get bf[idx], where idx is 0..3 *)
let get = fun bf -> fun idx ->
  let b0 = fst bf in
  let tail1 = snd bf in
  let b1 = fst tail1 in
  let tail2 = snd tail1 in
  let b2 = fst tail2 in
  let b3 = snd tail2 in
  if idx <= 0 then b0
  else if idx <= 1 then b1
  else if idx <= 2 then b2
  else b3 in

(* Insert element q into bf using hash
   functions h1 and h2 *)
let insert = fun q -> fun h1
            -> fun h2 -> fun bf ->
  let index1 = h1 q in
  let index2 = h2 q in
  let bf1 = set_true bf index1 in
  let bf2 = set_true bf1 index2 in
  bf2 in

(* Search for q in bf using hash
   functions h1 and h2 *)
let search = fun q -> fun h1
            -> fun h2 -> fun bf ->
  let index1 = h1 q in
  let index2 = h2 q in
  let b01 = get bf index1 in
  let b02 = get bf index2 in
  b01 && b02 in

(* Hash functions *)
let h1 = fun q ->
  if q <= -2 then 0
  else if -2 < q && q <= 0 then 1
  else if 0 < q && q <= 2 then 2
  else 3 in

let h2 = fun q ->
  if q <= -2 then 3
  else if -2 < q && q <= 0 then 2
  else if 0 < q && q <= 2 then 1
  else 0 in

(* Test whether the Bloom filter is full *)
let full = fun bf ->
  let b0 = fst bf in
  let t1 = snd bf in
  let b1 = fst t1 in
  let t2 = snd t1 in
  let b2 = fst t2 in
  let b3 = snd t2 in
  b0 && b1 && b2 && b3 in

(* Initial, empty Bloom filter *)
let init = (false, (false, (false, false))) in

(* Query key *)
let key = gaussian(0,1) in

(* Insert gaussian samples until bf is full
   or the key is reported present *)
let search_before_full = fix loop bf :=
  if full bf then bf
  else if search key h1 h2 bf then bf
  else
    let q = gaussian(0,1) in
    loop (insert q h1 h2 bf) in

(* Query whether search terminated before
   the filter became full *)
let bf_final = search_before_full init in
  not (full bf_final)
```

Fig. 23. A 4-bit Bloom filter program with continuous samples, locality-sensitive hash functions, and search and insert functions.

distinction preserves atoms, including singleton cells induced by the two cuts $< c$ and $\leq c$. The implementation currently provides this interface for Poisson and binomial distributions. Other

distributions such as multinomial require a different partition abstraction and are not supported by the real-line cut analysis.

This atomic-distribution interface is an implementation extension. The formal discretization and soundness arguments in this paper analyze the strict uniform family; they do not by themselves prove semantic preservation for the Poisson or binomial implementation paths. Extending the theorem requires an additional sampling case based on the tagged cumulative masses above.

The one-argument Laplace, Cauchy, Student-t, and logistic constructors are centered at zero. The exponential argument is a rate, while the gamma second argument is a scale. The names a, b in the Pareto, Weibull, and Gumbel rows follow the GSL routines called by the implementation. In particular, `gumbel1` and `gumbel2` are GSL parameterizations rather than location–scale constructors. For comparison, GSL's Gumbel-I parameters correspond to location $\mu = \log(b)/a$ and scale $1/a$, and its Gumbel-II law is a Fréchet law with shape a and scale $b^{1/a}$.

D Typing Rules for Full Slice Language

The typing rules for general language constructs is given in Figure 26.

E Discretization Proofs

E.1 Proof of Lemma 5.4

Theorem E.1 (Discretization Is Well-Typed). *Let $\Gamma \vdash e : \tau$. We have $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, where $\mathcal{D}[\![\Gamma]\!]$ transforms all types in Γ, i.e., $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ for all $x \in \mathrm{dom}(\Gamma)$.*

Proof. We proceed by induction on the structure of the typing derivation $\Gamma \vdash e : \tau$, matching each case of the discretization function $\mathcal{D}[\![\cdot]\!]$ defined in Figure 13.

Case $e = x$. Since $\Gamma(x) = \tau$, we have $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\tau]\!]$. Moreover, $\mathcal{D}[\![x : \tau]\!] = x$, so $\mathcal{D}[\![\Gamma]\!] \vdash x : \mathcal{D}[\![\tau]\!]$.

Case $e = (c : \mathbf{float}[B; V])$. The Float-Const rule was applied, so $\mathrm{has}(V, c)$ holds.

- If $B = \top$: then $\mathcal{D}[\![c : \mathbf{float}[B; V]]\!] = c$ and $\mathcal{D}[\![\mathbf{float}[B; V]]\!] = \mathbf{float}$. Since $\mathrm{has}(V, c)$ still holds, c is well-typed at $\mathbf{float}$ by Float-Const.
- If $B \neq \top$: then $\mathcal{D}[\![c : \mathbf{float}[B; V]]\!] = k_{\#|B|+1}$ for the unique k such that $c \in \mathrm{Intervals}(B)_k$, and $\mathcal{D}[\![\mathbf{float}[B; V]]\!] = \mathbf{fin}(|B| + 1)$. Since $0 \leq k < |B| + 1$, the finite constant $k_{\#|B|+1}$ is well-typed at $\mathbf{fin}(|B| + 1)$ by FinConst.

Case $e = (\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2)$. By the inductive hypothesis,

$$\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_1 : \tau_1]\!] : \mathcal{D}[\![\tau_1]\!] \quad \text{and} \quad \mathcal{D}[\![\Gamma, x : \tau_1]\!] \vdash \mathcal{D}[\![e_2 : \tau_2]\!] : \mathcal{D}[\![\tau_2]\!].$$

Note that $\mathcal{D}[\![\Gamma, x : \tau_1]\!] = \mathcal{D}[\![\Gamma]\!], x : \mathcal{D}[\![\tau_1]\!]$. Applying Let gives

$$\mathcal{D}[\![\Gamma]\!] \vdash \mathbf{let}\ x = \mathcal{D}[\![e_1 : \tau_1]\!]\ \mathbf{in}\ \mathcal{D}[\![e_2 : \tau_2]\!] : \mathcal{D}[\![\tau_2]\!].$$

Case $e = (\mathbf{if}\ e_1 : \mathbf{bool}\ \mathbf{then}\ e_2 : \tau\ \mathbf{else}\ e_3 : \tau)$. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \tau_i]\!] : \mathcal{D}[\![\tau_i]\!]$ for $i = 1, 2, 3$, where $\tau_1 = \mathbf{bool}$ and $\tau_2 = \tau_3 = \tau$. Since $\mathcal{D}[\![\mathbf{bool}]\!] = \mathbf{bool}$, applying If gives

$$\mathcal{D}[\![\Gamma]\!] \vdash \mathbf{if}\ \mathcal{D}[\![e_1 : \mathbf{bool}]\!]\ \mathbf{then}\ \mathcal{D}[\![e_2 : \tau]\!]\ \mathbf{else}\ \mathcal{D}[\![e_3 : \tau]\!] : \mathcal{D}[\![\tau]\!].$$

Case $e = (e_1 : \mathbf{float}[B; V_1] < e_2 : \mathbf{float}[B; V_2])$. The Less rule was applied, so both subexpressions share the same cut set B and $\mathrm{answers}_<(B, V_1, V_2)$ holds. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \mathbf{float}[B; V_i]]\!] : \mathcal{D}[\![\mathbf{float}[B; V_i]]\!]$ for $i = 1, 2$.

- If $B = \top$: then $\mathcal{D}[\![\mathbf{float}[B; V_i]]\!] = \mathbf{float}$ for both i, and the standard float comparison gives type $\mathbf{bool} = \mathcal{D}[\![\mathbf{bool}]\!]$.
- If $B \neq \top$: then both subexpressions are discretized to type $\mathbf{fin}(|B| + 1)$, and the finite comparison $<_{\#(|B|+1)}$ is well-typed at $\mathbf{bool} = \mathcal{D}[\![\mathbf{bool}]\!]$ by FinLess.

Case $e = (e_1 : \mathbf{float}[B; V_1] \leq e_2 : \mathbf{float}[B; V_2])$. Analogous to the $<$ case, using $\mathrm{answers}_\leq(B, V_1, V_2)$ and FinLessEq.

Case $e = (\mathbf{uniform}(e_1 : \mathbf{float}[B_1; V_1],\ e_2 : \mathbf{float}[B_2; V_2]) : \mathbf{float}[B; \top])$. The Continuous-2 rule was applied, so $\mathrm{recovers}(B_1, V_1)$ and $\mathrm{recovers}(B_2, V_2)$ hold. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \mathbf{float}[B_i; V_i]]\!] : \mathcal{D}[\![\mathbf{float}[B_i; V_i]]\!]$ for $i = 1, 2$. We consider each sub-case of the discretization function:

- $e_1, e_2 \in \mathbb{R}$ and $e_1 \geq e_2$: then $\mathcal{D}[\![\mathbf{uniform}(\ldots) : \mathbf{float}[B; \top]]\!] = \mathbf{diverge}$, which is well-typed at any type by Diverge.

Source constructor	Density or PMF	Support	Parameter domain and convention
`exponential(`λ`)`	$\lambda e^{-\lambda x}$	$[0, \infty)$	Rate $\lambda > 0$
`uniform(`a, b`)`	$\frac{1}{b-a}\mathbf{1}_{[a,b)}(x)$	$[a, b]$	Endpoints $a < b$
`laplace(`b`)`	$\frac{1}{2b}e^{-\lvert x\rvert/b}$	$\mathbb{R}$	Centered; scale $b > 0$
`gaussian(`μ, σ`)`	$\frac{1}{\sqrt{2\pi}\sigma}e^{-(x-\mu)^2/(2\sigma^2)}$	$\mathbb{R}$	Location $\mu \in \mathbb{R}$; standard deviation $\sigma > 0$
`cauchy(`γ`)`	$\frac{\gamma}{\pi(x^2+\gamma^2)}$	$\mathbb{R}$	Centered; scale $\gamma > 0$
`beta(`α, β`)`	$\frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha,\beta)}$	$[0, 1]$	Shapes $\alpha, \beta > 0$
`tdist(`ν`)`	$\frac{\Gamma((\nu+1)/2)}{\sqrt{\nu\pi}\,\Gamma(\nu/2)}(1 + x^2/\nu)^{-(\nu+1)/2}$	$\mathbb{R}$	Centered; degrees of freedom $\nu > 0$
`lognormal(`μ, σ`)`	$\frac{1}{x\sigma\sqrt{2\pi}}e^{-(\log x-\mu)^2/(2\sigma^2)}$	$[0, \infty)$	Log-location $\mu \in \mathbb{R}$; log-scale $\sigma > 0$
`chi2(`ν`)`	$\frac{x^{\nu/2-1}e^{-x/2}}{2^{\nu/2}\Gamma(\nu/2)}$	$[0, \infty)$	Real degrees of freedom $\nu > 0$
`gamma(`α, θ`)`	$\frac{x^{\alpha-1}e^{-x/\theta}}{\Gamma(\alpha)\theta^{\alpha}}$	$[0, \infty)$	Shape $\alpha > 0$; scale $\theta > 0$
`logistic(`s`)`	$\frac{e^{-x/s}}{s(1+e^{-x/s})^2}$	$\mathbb{R}$	Centered; scale $s > 0$
`pareto(`a, b`)`	$ab^a x^{-a-1}$	$[b, \infty)$	GSL shape $a > 0$; threshold/scale $b > 0$
`rayleigh(`σ`)`	$\frac{x}{\sigma^2}e^{-x^2/(2\sigma^2)}$	$[0, \infty)$	Scale $\sigma > 0$
`weibull(`a, b`)`	$\frac{b}{a}(x/a)^{b-1}e^{-(x/a)^b}$	$[0, \infty)$	GSL scale $a > 0$; exponent $b > 0$
`gumbel1(`a, b`)`	$ab\,e^{-(be^{-ax}+ax)}$	$\mathbb{R}$	GSL parameters $a, b > 0$
`gumbel2(`a, b`)`	$ab\,x^{-a-1}e^{-bx^{-a}}$	$[0, \infty)$	GSL shape $a > 0$; coefficient $b > 0$
`exppow(`a, b`)`	$\frac{1}{2a\Gamma(1+1/b)}e^{-(\lvert x\rvert/a)^b}$	$\mathbb{R}$	Scale $a > 0$; exponent $b > 0$
`poisson(`μ`)`	$e^{-\mu}\mu^k/k!$	$\mathbb{N}_0$ if $\mu > 0$; $\{0\}$ if $\mu = 0$	Mean $\mu \geq 0$
`binomial(`p, n`)`	$\binom{n}{k}p^k(1-p)^{n-k}$	$\{0, \ldots, n\}$, with endpoint degeneracies	$0 \leq p \leq 1$; integer $0 \leq n \leq \min(2^{32} - 1, \texttt{max_int})$

Fig. 25. Implemented scalar distribution laws and dynamic parameter domains. The binomial support is $\{0\}$ when $n = 0$ or $p = 0$, and $\{n\}$ when $p = 1$; otherwise it is $\{0, \ldots, n\}$.